

**Asymptotická výpočtová zložitosť, notácia veľké O,
amortizovaná zložitosť.**

Výpočtová zložitosť má dve základné miery:

- časovú,
- pamäťovú.

Časová zložitosť algoritmu je funkcia, ktorá vyjadruje najhorší časový odhad trvania behu nejakého algoritmu, teda koľko vykoná elementárnych operácií vzhľadom k množine vstupných dát.

Asymptotická zložitosť sa používa pri klasifikácii algoritmov, čo je rozdelenie algoritmov do tried zložítostí, u ktorých platí, že od určitej veľkosti dát je algoritmus danej triedy vždy pomalší ako algoritmus triedy predchádzajúcej. Ak sú dva algoritmy v rôznych triedach asymptotickej zložítosti, tak vždy existuje také množstvo dát, od ktorého je asymptoticky lepší algoritmus vždy rýchlejší bez ohľadu na to, koľkokrát je niektorý z počítačov rýchlejší.

Elementárne operácie sú:

- priradenie do premennej `prem = hodnota`,
- zistenie hodnoty premennej `prem`,
- vykonanie aritmetickej operácie `hodn1 op hodn2`,
- porovnanie dvoch hodnôt `hodn1 rel hodn2`,
- zaindexovanie prvku poľa `prem[index]`,
- volanie funkcie,
- návrat z funkcie.

Ako si zjednodušiť zisťovanie časovej zložítosti:

- postupnosť príkazov, v ktorej je každý $O(1)$, má zložitosť $O(1)$,
- for cyklus s n -prechodmi, ktorý vykonáva len $O(1)$, má zložitosť $O(n)$,
- všeobecnejšie for-cyklus s n -prechodmi, ktorý vykonáva telo cyklu $O(f)$, má zložitosť $O(n * f)$,
- while cyklus, ktorý znižuje (zvyšuje) nejakú testovanú premennú o konštantu, má zložitosť $O(n * f)$, kde $O(f)$ je telo cyklu, n je testovaná hodnota,
- while cyklus, ktorý delí nejakú testovanú premennú o konštantu (napr. na polovicu), má zložitosť $O(\log n * f)$, kde $O(f)$ je telo cyklu, n je testovaná hodnota,
- ak máme spolu dva algoritmy, pri ktorých sa druhý vykoná po skončení prvého, potom majú celkovú zložitosť $O(f_1 + f_2)$,
- rekurzívne funkcie majú často zložitosť rádovo porovnateľnú s počtom vygenerovaných výsledkov.

Notácia veľké O

Funkcie časovej zložitosti zaradíme do tried tak, že každá z tried bude charakterizovaná nejakou funkciou, ktorá ju ohraničuje zhora.

Označenie: $f(n) \in O(g(n))$

Definícia: Existuje $c > 0$ a $n_0 > 0$ také, že $(\forall n > n_0) (0 \leq f(n) \leq c \cdot g(n))$

Pre dané dve funkcie $f(n)$ a $g(n)$ hovoríme, že $f(n)$ je triedy $O(g(n))$, ak existujú také dve kladné konštanty c a n_0 , že pre každé $n > n_0$ platí: $0 \leq f(n) \leq c \cdot g(n)$.

Funkcii $g(n)$ sa hovorí **horný asymptotický odhad** funkcie $f(n)$.

Základné triedy funkcií časovej zložitosti

$O(1)$ konštantná zložitosť

- nezávisí od veľkosti vstupu, výstup nejakej funkcie podľa vzorca

$O(\log n)$ logaritmická zložitosť

- príkladom algoritmu s takou zložitou je hľadanie prvku v utriedenom poli binárnym vyhľadávaním

$O(n)$ lineárna zložitosť

- napríklad hľadanie prvku v neutriedenom poli

$O(n \log n)$ zložitosť $n \cdot \log n$

- najrýchlejšie triediace algoritmy s prvkami ľubovoľných typov majú túto zložitosť
- merge sort, quick sort (ak zlý pivot, tak $O(n^2)$)

$O(n^2)$ kvadratická zložitosť

- bublinkové triedenie

$O(n^3)$ kubická zložitosť

- napríklad Floyd-Warshallov algoritmus hľadania najkratšej cesty v ohodnotenom grafe

$O(2^n)$ exponenciálna zložitosť

- backtracking, čiže algoritmy prehľadávania s návratom

$O(n!)$ faktoriálová zložitosť

- algoritmus pre rekurzívne generovanie všetkých permutácií

Porovnanie

Zrejme medzi týmito triedami zložitosti platí takýto vzťah:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Amortizovaná časová zložitosť

Asymptotická zložitosť ukazuje najhorší možný prípad inštancie behu algoritmu, čo ale nie je vždy vypovedajúce. Aj tá najhoršia sekvencia prípadov môže mať za určitých okolností výrazne lepší priebeh, než by asymptotická zložitosť napovedala. Tento zdanlivý paradox je zapríčinený tým, že operácie s vysokou zložitosťou zmenia dátovú štruktúru tak, že takto zlý prípad nenastane nejakú dlhšiu dobu – tým sa zložitá operácia amortizuje.

Ukážeme si to na príklad v jazyku python:

- chceme implementovať pythonovské pole - *list()*,
- zadefinujeme triedu *DynamickéPole*, ktorá využíva na implementáciu modul *ctypes* jazyka C,
- metóda *append*, ktorá okrem pridania nového prvku do poľa, bude naše pole nafukovať napr. vždy o dvojnásobok momentálnej dĺžky poľa (ak by sme zväčšovali len o nejakú pevne stanovenú konštantu, amortizovaná zložitosť by bola $O(n)$),
- pri mocninách 2 sa pole nafúkne a medzi tým je veľakrát zložitosť $O(1)$, čiže jednu časovo náročnú operáciu vieme rozložiť k rýchlym operáciám, pri ktorých si ušetríme – amortizujeme – dosť, aby sme si raz začas mohli dovoliť jednu časovo náročnú operáciu,
- vďaka tomuto mechanizmu má „priemerný“ čas operácie *append* zložitosť $O(1)^*$, tzv. amortizovanú zložitosť.